

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **C++ Programming**

Practical Assignment - I

- Q1. Write a Program to Print ***“Your Name”*** and ***“Your College Name”*** at 2 different lines.
- Q2. Write a Program to read a number and find its square.
- Q3. Write a Program to find Area of Circle.
- Q4. Write a Program to calculate total & percentage from 3 subjects.
- Q5. Write a Program to check given number is Even or Odd.
- Q6. Write a Program to accept two integers and check whether they are equal or not.
- Q7. Write a Program to find sum and average of 3 numbers given from user.
- Q8. Write a Program to check whether an alphabet is a vowel or not.
- Q9. Write a program to read the age of a candidate and determine whether he is eligible for vote or not.
- Q10. Write a program to find the largest number among three numbers.
- Q11. Write a Program to find the factorial of a given number.
- Q12. Write a Program to generate table of a number entered by user.
- Q13. Write a program to find sum of digits of given numbers.
- Q14. Write a program to read and print data of student using class and object.
- Q15. Write a program to initialize the data members of object using constructor.
- Q16. Write a Program to find the sum of 2, 3 and 4 integer numbers using function overloading.
- Q17. Write a Program to create array of objects for Students.
- Q18. Write a Program for Friend Function.
- Q19. Write a Program to overload Unary Operator.
- Q20. Write a Program to overload Binary Operator.

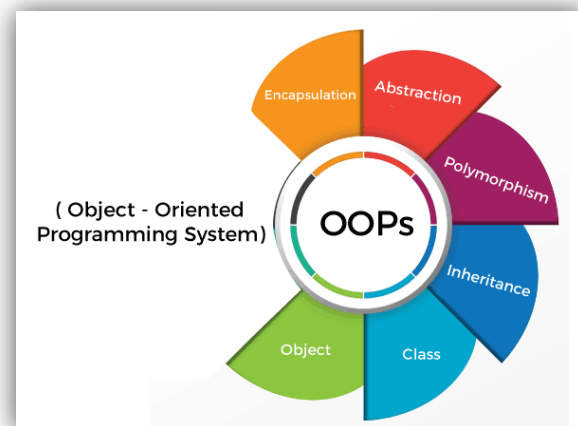
Object Oriented Programming Paradigm-

Paradigm can also be termed as method to solve some problem or do some task. Programming paradigm is an approach to solve problem using some programming language or we can say it is a method to solve a problem using tools and techniques that are available to us following some approach. There are lots for programming language that are known but all of them need to follow some strategy when they are implemented and this methodology is paradigms. Apart from varieties of programming language there are lots of paradigms to fulfill each and every demand.

The program is written as a collection of classes and object which are meant for communication. The smallest and basic entity is object and all kind of computation is performed on the objects only. More emphasis is on data rather procedure. It can handle almost all kind of real life problems which are today in scenario.

Advantages:

- Data security
- Inheritance
- Code reusability
- Flexible and abstraction is also present



Basic concept of OOP:

Object oriented programming is a type of programming which uses objects and classes its functioning. The object oriented programming is based on real world entities like inheritance, polymorphism, data hiding, etc. It aims at binding together data and function work on these data sets into a single entity to restrict their usage.

Some basic concepts of object oriented programming are –

1. CLASS
2. OBJECTS
3. ENCAPSULATION
4. POLYMORPHISM
5. INHERITANCE
6. ABSTRACTION

[Principles of Object Oriented Programming]

Object

An object is a real-world element in an object-oriented environment that may have a physical or a conceptual existence. Each object has –

-  Identity that distinguishes it from other objects in the system.
-  State that determines characteristic properties of an object as well as values of properties that the object holds.
-  Behavior that represents externally visible activities performed by an object in terms of changes in its state.

Objects can be modeled according to the needs of the application. An object may have a physical existence, like a customer, a car, etc.; or an intangible conceptual existence, like a project, a process, etc.

Class

A class represents a collection of objects having same characteristic properties that exhibit common behavior. It gives the blueprint or the description of the objects that can be created from it. Creation of an object as a member of a class is called instantiation. Thus, an object is an instance of a class.

The constituents of a class are –

-  A set of attributes for the objects that are to be instantiated from the class. Generally, different objects of a class have some difference in the values of the attributes. Attributes are often referred as class data.
-  A set of operations that portray the behavior of the objects of the class. Operations are also referred as functions or methods.

Encapsulation

Encapsulation is the process of binding both attributes and methods together within a class. Through encapsulation, the internal details of a class can be hidden from outside. It permits the elements of the class to be accessed from outside only through the interface provided by the class.

Polymorphism

Polymorphism is originally a Greek word that means the ability to take multiple forms. In object-oriented paradigm, polymorphism implies using operations in different ways, depending upon the instances they are operating upon. Polymorphism allows objects with different internal structures to have a common external interface. Polymorphism is particularly effective while implementing inheritance.

Inheritance

Inheritance is an important pillar of Object-Oriented Programming. The capability of a class to derive properties and characteristics from another class is called Inheritance. When we write a class, we inherit properties from other classes. So when we create a class, we do not need to write all the properties and functions again and again, as these can be inherited from another class that possesses it. Inheritance allows the user to reuse the code whenever possible and reduce its redundancy.

Data Abstraction

Data abstraction is one of the most essential and important features of object-oriented programming. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation. Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

Dynamic Binding

In dynamic binding, the code to be executed in response to the function call is decided at runtime. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time. Dynamic Method Binding One of the main advantages of inheritance is that some derived class D has all the members of its base class B. Once D is not hiding any of the public members of B, then an object of D can represent B in any context where a B could be used. This feature is known as subtype polymorphism.

Message Passing

It is a form of communication used in object-oriented programming as well as parallel programming. Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function, and the information to be sent.

Comparison of Procedural Programming and OOP

Object-Oriented Programming (OOP)

- OOP treats data as a critical element in the program development and does not allow it to flow freely around the system.
- In OOP, the major emphasis is on data rather than procedure (function).
- It ties data more closely to the function that operate on it, and protects it from accidental modification from outside function.
- OOP allows decomposition of a problem into a number of entities called objects and then builds data and function around these objects.
- The data of an object can be accessed only by the function associated with that object. However, function of one object can access the function of other objects.
- C++, Java, Dot Net, Python etc are the example of Object oriented programming (OOP) language.

Some Characteristics of Object Oriented Programming:-

- Emphasis is on data rather than procedure (function).
- Programs are divided into objects.
- Functions that operate on the data of an object are ties together in the data structure.
- Data is hidden and cannot be accessed by external function.
- Objects may communicate with each other through function.
- New data and functions can be easily added whenever necessary.
- Follows bottom up approach in program design.

Application of Object Oriented Programming:-

- User interface design such as windows, menu
- Real Time Systems such as Control system for cars, aircraft, space vehicles etc
- Office automation system such as Document Management System i.e. Word processing system, spread sheet software etc
- AI and Expert System
- Neural Networks and parallel programming System
- Decision support system etc

Advantages of OOP:-

1. Improved software-development productivity:

Object-oriented programming is modular, as it provides separation of duties in object-based program development. It is also extensible, as objects can be extended to include new attributes and behaviors. Objects can also be reused within an across applications. Because of these three factors – modularity, extensibility, and reusability – object-oriented programming provides improved software-development productivity over traditional procedure-based programming techniques.

2. Improved software maintainability:

Since the design is modular, part of the system can be updated in case of issues without a need to make large-scale changes.

3. Faster development:

Reuse enables faster development. Object-oriented programming languages come with rich libraries of objects, and code developed during projects is also reusable in future projects.

4. Lower cost of development:

The reuse of software also lowers the cost of development. Typically, more effort is put into the object-oriented analysis and design, which lowers the overall cost of development.

5. Higher-quality software:

Faster development of software and lower cost of development allows more time and resources to be used in the verification of the software. Although quality is dependent upon the experience of the teams, object oriented programming tends to result in higher-quality software.

Disadvantages of OOP:-

1. Larger program Size:

Object oriented programs are much larger than other programs. In the early days of computing, space on hard drives, floppy drives and in memory was at a premium. Today we do not have these restrictions.

2. Effort:

Object oriented programs require a lot of work to create. Specifically, a great deal of planning goes into an object oriented program well before a single piece of code is ever written. Initially, this early effort was felt by many to be a waste of time. In addition, because the programs were larger, coders spent more time actually writing the program.

3. Speed:

Object oriented programs are slower than other programs, partially because of their size. Other aspects of Object Oriented Programs also demand more system resources, thus slowing the program down.

Procedural Oriented Programming (POP)

- In the procedure oriented approach, large programs are divided into smaller programs known as functions.
- In POP, a program is written as a sequence of procedures or function.
- In POP, each procedure (function) contains a series of instructions for performing a specific task.
- During the program execution each procedure (function) can be called by the other procedures.
- To call a procedure (function), we have to write function name only.
- While we concentrate onto the development of functions, we give very little attention to the data that are being used by various functions.
- In POP, the major emphasis is on procedure (function) and not on the data.
- In a multi-function program, many important data items are placed as global so that they may be accessed by all the functions. Each function may have its own local data.
- Global data are more vulnerable to an accidental change by a function. In a large program it is very difficult to identify what data is used by which function.

[Principles of Object Oriented Programming]

- Examples of procedural oriented programming language are COBOL, FORTRAN, PASCAL, C programming language etc.

Characteristics of procedure-oriented programming:-

- Large programs are divided into smaller programs known as functions.
- Most of the functions share global data.
- Data move openly around the system from function to function.
- Functions change the value of data at any time from any place. (Functions transform data from one form to another.)
- It uses top-down approach in program design.

Drawback:-

In POP, global data can be accessed & changed by any procedure (function) so there is no data security. In case if we want to change type of data of global data, then we also need to resolve all functions that access the data. Due to this it may happen that some errors will occur.

POP does not model real world problems because functions are action oriented.

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: Object Oriented Programming With C++

Unit I

[Principles of Object Oriented Programming]

S.no.	On the basis of	Procedural Programming	Object-oriented programming
1.	Definition	It is a programming language that is derived from structure programming and based upon the concept of calling procedures. It follows a step-by-step approach in order to break down a task into a set of variables and routines via a sequence of instructions.	Object-oriented programming is a computer programming design philosophy or methodology that organizes/ models software design around data or objects rather than functions and logic.
2.	Security	It is less secure than OOPs.	Data hiding is possible in object-oriented programming due to abstraction. So, it is more secure than procedural programming.
3.	Approach	It follows a top-down approach.	It follows a bottom-up approach.
4.	Data movement	In procedural programming, data moves freely within the system from one function to another.	In OOP, objects can move and communicate with each other via member functions.
5.	Orientation	It is structure/procedure-oriented.	It is object-oriented.
6.	Access modifiers	There are no access modifiers in procedural programming.	The access modifiers in OOP are named as private, public, and protected.
7.	Inheritance	Procedural programming does not have the concept of inheritance.	There is a feature of inheritance in object-oriented programming.
8.	Code reusability	There is no code reusability present in procedural programming.	It offers code reusability by using the feature of inheritance.
9.	Overloading	Overloading is not possible in procedural programming.	In OOP, there is a concept of function overloading and operator overloading.
10.	Importance	It gives importance to functions over data.	It gives importance to data over functions.
11.	Virtual class	In procedural programming, there are no virtual classes.	In OOP, there is an appearance of virtual classes in inheritance.

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: Object Oriented Programming With C++

Unit I

[Principles of Object Oriented Programming]

12.	Complex problems	It is not appropriate for complex problems.	It is appropriate for complex problems.
13.	Data hiding	There is not any proper way for data hiding.	There is a possibility of data hiding.
14.	Program division	In Procedural programming, a program is divided into small programs that are referred to as functions.	In OOP, a program is divided into small parts that are referred to as objects.
15.	Examples	Examples of Procedural programming include C, Fortran, Pascal, and VB.	The examples of object-oriented programming are - .NET, C#, Python, Java, VB.NET, and C++.

Difference between C and C++

What is C?

C is a structural or procedural oriented programming language which is machine-independent and extensively used in various applications.

C is the basic programming language that can be used to develop from the operating systems (like Windows) to complex programs like Oracle database, Git, Python interpreter, and many more. C programming language can be called a god's programming language as it forms the base for other programming languages. If we know the C language, then we can easily learn other programming languages. C language was developed by the great computer scientist Dennis Ritchie at the Bell Laboratories. It contains some additional features that make it unique from other programming languages.

What is C++?

C++ is a special-purpose programming language developed by Bjarne Stroustrup at Bell Labs circa 1980. C++ language is very similar to C language, and it is so compatible with C that it can run 99% of C programs without changing any source of code though C++ is an object-oriented programming language, so it is safer and well-structured programming language than C.

The following are the differences between C and C++:

Definition

C is a structural programming language, and it does not support classes and objects, while C++ is an object-oriented programming language that supports the concept of classes and objects.

Type of programming language

C supports the structural programming language where the code is checked line by line, while C++ is an object-oriented programming language that supports the concept of classes and objects.

Developer of the language

Dennis Ritchie developed C language at Bell Laboratories while Bjarne Stroustrup developed the C++ language at Bell Labs circa 1980.

Subset

C++ is a superset of C programming language. C++ can run 99% of C code but C language cannot run C++ code.

Type of approach

C follows the top-down approach, while C++ follows the bottom-up approach. The top-down approach breaks the main modules into tasks; these tasks are broken into sub-tasks, and so on. The bottom-down approach develops the lower level modules first and then the next level modules.

Security

In C, the data can be easily manipulated by the outsiders as it does not support the encapsulation and information hiding while C++ is a very secure language, i.e., no outsiders can manipulate its data as it supports both encapsulation and data hiding. In C language, functions and data are the free entities, and in C++ language, all the functions and data are encapsulated in the form of objects.

Function Overloading

Function overloading is a feature that allows you to have more than one function with the same name but varies in the parameters. C does not support the function overloading, while C++ supports the function overloading.

Function Overriding

[Principles of Object Oriented Programming]

Function overriding is a feature that provides the specific implementation to the function, which is already defined in the base class. C does not support the function overriding, while C++ supports the function overriding.

Reference variables

C does not support the reference variables, while C++ supports the reference variables.

Keywords

C contains 32 keywords, and C++ supports 52 keywords.

Namespace feature

A namespace is a feature that groups the entities like classes, objects, and functions under some specific name. C does not contain the namespace feature, while C++ supports the namespace feature that avoids the name collisions.

Exception handling

C does not provide direct support to the exception handling; it needs to use functions that support exception handling. C++ provides direct support to exception handling by using a try-catch block.

Input/Output functions

In C, scanf and printf functions are used for input and output operations, respectively, while in C++, cin and cout are used for input and output operations, respectively.

Memory allocation and de-allocation

C supports calloc() and malloc() functions for the memory allocation, and free() function for the memory de-allocation. C++ supports a new operator for the memory allocation and delete operator for the memory de-allocation.

Inheritance

Inheritance is a feature that allows the child class to reuse the properties of the parent class. C language does not support the inheritance while C++ supports the inheritance.

Header file

C program uses <stdio.h> header file while C++ program uses <iostream.h> header file.

C++ Token:

A C++ program is composed of tokens which are the smallest individual unit. Tokens can be one of several things, including keywords, identifiers, constants, operators, or punctuation marks.

There are 6 types of tokens in c++:

- Keyword
- Identifiers
- Constants
- Strings
- Special symbols
- Operators

1. Keywords:

In C++, keywords are reserved words that have a specific meaning in the language and cannot be used as identifiers. Keywords are an essential part of the C++ language and are used to perform specific tasks or operations.

Some of the keywords in c++ are:

Class	Auto	static	extern
register	public	private	protected
try	catch	throw	const
friend	Inline	new	delete

2. Identifiers

In C++, an identifier is a name given to a variable, function, or another object in the code. Identifiers are used to refer to these entities in the program, and they can consist of letters, digits, and underscores. However, some rules must be followed when choosing an identifier:

- The first character must be a letter or an underscore.
- Identifiers cannot be the same as a keyword.
- Identifiers cannot contain any spaces or special characters except for the underscore.
- Identifiers are case-sensitive, meaning that a variable named "myVariable" is different from a variable named "myvariable".

Valid: my_variable, student_name, balance_due

Invalid:

my variable (contains a space)

student# (contains a special character)

int (same as a keyword)

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **Object Oriented Programming using C++**

Unit II

3. Constants

In C++, a constant is a value that cannot be changed during the execution of the program. Constants often represent fixed values used frequently in the code, such as the value of pi or the maximum size of an array.

To define a constant in C++, you use the `const` keyword followed by the data type and the constant's name and then initialize it with a value. The value of a constant cannot be changed once it has been defined. Here is an example:

```
const double PI = 3.14159;
```

4. String:

In C++, a string is a sequence of characters that represents text. Strings are commonly used in C++ programs to store and manipulate text data. To use strings in C++, you must include the `<string>` header file at the beginning of your program.

5. Special symbols:

In C++, several special symbols are used for various purposes. The ampersand (&) is used to represent the address of a variable, while the tilde (~) is used to represent the bitwise NOT operator. Some of the most common special symbols include the asterisk (*), used as the multiplication and pointer operators.

The pound sign (#) is used to represent the preprocessor directive, a special type of instruction processed by the preprocessor before the code is compiled. The percent sign (%) is used as the modulus operator, which is used to find the remainder when one number is divided by another.

The vertical bar (|) is used to represent the bitwise OR operator, while the caret (^) is used to represent the bitwise XOR operator. The exclamation point (!) is used to represent the logical NOT operator, which is used to negate a Boolean value.

In addition to these symbols, C++ also has a number of other special characters that are used for a variety of purposes. For example, the backslash (\) is used to escape special characters, the double quotes (") are used to enclose string literals, and the single quotes (') are used to enclose character literals.

Overall, the special symbols in C++ are an important part of the language and are used in a variety of different contexts to perform a wide range of operations.

6. Operators:

In C++, operators are special symbols that are used to perform operations on one or more operands. An operand is a value on which an operator acts. C++ has a wide range of operators, including arithmetic operators, relational operators, logical operators, and others.

C++ Character Set:

Character set is the combination of English language (Alphabets and White spaces) and math's symbols (Digits and Special symbols). Character Set means that the characters and symbols that a C++ Program can understand and accept. These are grouped to form the commands, expressions, words, c-statements and other tokens for C++ Language.

C++ Character set is basically a set of valid characters that convey a specific connotation to the compiler. We use characters to represent letters, digits, special symbols, white spaces, and other characters.

The C++ character set consists of following main elements. They are:

1. Alphabets

Alphabets are represented by A-Z or a-z. C- Language is case sensitive so it takes different meaning for small and upper case letters. By using this character set C statements and character constants can be written very easily. There are total 26 letters used in C-programming.

2. Digits

Digits are represented by 0-9 or by combination of these digits. By using the digits numeric constant can be written easily. Also numeric data can be assigned to the C-tokens. There are total 10 digits used in the C-programming.

3. Special Symbols

All the keyboard keys except alphabet, digits and white spaces are the special symbols. These are some punctuation marks and some special symbols used for special purpose.

There are total 30 special symbols used in the C-programming. Special symbols are used for C-statements like to create an arithmetic statement +, -, * etc. , to create relational statement <, >, <=, >=, == etc. , to create assignment statement =, to create logical statement &&, || etc. are required.

4. White Spaces-

White spaces has blank space, new line return, Horizontal tab space, carriage ctrl, Form feed etc. are all used for special purpose. Also note that Turbo-C Compiler always ignore these white space characters in both high level and low level programming.

Symbolic Constants in C++:

The symbolic constants are constant identifier. Before the symbolic constants are used they be initialized. So, after a symbolic constant is initialized as per the rules, its value cannot be changed during program execution. Every programming language has its own style of declaring the constants, in C++ programming language the symbolic constants can be declared in two ways:

1. Using 'const' keyword
2. Using 'define' directive

The 'const' Keyword in C++:

The 'const' keyword is used to declare constant identifier. The constant identifier is declared in the same exact way as variable is declared except that a keyword "const" is used to specify a constant identifier. A value must be assigned to constant identifier at the time of its declaration. Once a value is assigned, it cannot be changed during program execution. For example, to declare a variable PI and to assign value 3.1417 by using 'const' keyword, the statement is written as:

const float PI=3.1417;

PI is the variable name, its type is float and this is a constant. So, currently a value of 3.1417 is stored in the variable PI. This value will remain the same throughout the program.

```
int main()
{
    float R,H, volume;
    const float PI=3.1417;
    cout<<"Enter the value of R: ";
    cin>>R;
    cout<<"Enter the value of H: ";
    cin>>H;
    volume=PI*R*R*H;
    cout<<"Volume of a Cylinder is = "<<volume;
}
```

The 'define' Directive:

The 'define' is a preprocessor directive. It is used to define a constant identifier known as constant macro. A constant macro is an identifier, which is assigned a particular constant in C++ value. like other preprocessor directives, the define directive is also used at the beginning of the program.

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **Object Oriented Programming using C++**

Unit II

The general syntax of 'define' directive is as follows:

#define identifier expression;

Identifier:

It indicates the name of constant identifier to which the constant value is to be assigned. This name cannot be used again in the program as variable name or function name etc.

Expression:

It indicates the constant value that is to be assigned to the identifier. It may be a constant value, a string or an arithmetic expression.

In the following statement, PI is assigned value 3.141592.

#define PI 3.141593

Difference Between define and const in C++:

The difference between 'define' directive and 'const' qualifier is as follows:

Define Directive	const Qualifier
It is used as preprocessor directive.	It is used as statement.
It is not terminated with semicolon (;)	It is terminated with semicolon.
Data type of constant identifier is not specified	Data type of constant identifier is specified.

C++ Memory Management: new and delete

C++ allows us to allocate the memory of a variable or an array in run time. This is known as dynamic memory allocation.

In C++, we need to deallocate the dynamically allocated memory manually after we have no use for the variable.

We can allocate and then deallocate memory dynamically using the new and delete operators respectively.

new operator-

Both the malloc() and new in C++ are used for the same purpose. They are used for allocating memory at the runtime. But, malloc() and new have different syntax. The main difference between the malloc() and new is that the new is an operator while malloc() is a standard library function that is predefined in a stdlib header file.

The new is a memory allocation operator, which is used to allocate the memory at the runtime. The memory initialized by the new operator is allocated in a heap. It returns the starting address of the memory, which gets assigned to the variable. The functionality of the new operator in C++ is similar to the malloc() function, which was used in the C programming language. C++ is compatible with the malloc() function also, but the new operator is mostly used because of its advantages.

Syntax of new operator

type variable = new type(parameter_list);

- **type:** *It defines the datatype of the variable for which the memory is allocated by the new operator.*
- **variable:** *It is the name of the variable that points to the memory.*
- **parameter_list:** *It is the list of values that are initialized to a variable.*

The new operator does not use the sizeof() operator to allocate the memory. It also does not use the resize as the new operator allocates sufficient memory for an object. It is a construct that calls the constructor at the time of declaration to initialize an object.

As we know that the new operator allocates the memory in a heap; if the memory is not available in a heap and the new operator tries to allocate the memory, then the exception is thrown. If our code is not able to handle the exception, then the program will be terminated abnormally.

delete operator-

Once we no longer need to use a variable that we have declared dynamically, we can deallocate the memory occupied by the variable.

For this, the delete operator is used. It returns the memory to the operating system. This is known as memory deallocation.

Syntax of delete operator

We can delete a specific element or variable using the delete operator, as shown:

```
delete pointer_variable;
```

```
// delete ptr;
```

It deallocates memory for one element

Similarly, we can delete the block of allocated memory space using the delete [] operator.

```
delete [ ] pointer_variable;
```

```
// delete [] ptr; It deallocate for an array
```

References in C++ / (Reference Variable)

When a variable is declared as a reference, it becomes an alternative name for an existing variable. A variable can be declared as a reference by putting ‘&’ in the declaration.

Also, we can define a reference variable as a type of variable that can act as a reference to another variable. ‘&’ is used for signifying the address of a variable or any memory. Variables associated with reference variables can be accessed either by its name or by the reference variable associated with it.

```
// C++ Program to demonstrate
```

```
// use of references
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int x = 10;
```

```
    // ref is a reference to x.
```

```
    int& ref = x;
```

```
// Value of x is now changed to 20
ref = 20;
cout << "x = " << x << '\n';

// Value of x is now changed to 30
x = 30;
cout << "ref = " << ref << '\n';

return 0;
}
```

A reference variable is an alias, that is, another name for an already existing variable. Once a reference is initialized with a variable, either the variable name or the reference name may be used to refer to the variable.

References vs Pointers

References are often confused with pointers but three major differences between references and pointers are –

-  You cannot have NULL references. You must always be able to assume that a reference is connected to a legitimate piece of storage.
-  Once a reference is initialized to an object, it cannot be changed to refer to another object. Pointers can be pointed to another object at any time.
-  A reference must be initialized when it is created. Pointers can be initialized at any time.

Basic Data Type of C++:

All variables use data type during declaration to restrict the type of data to be stored. Therefore, we can say that data types are used to tell the variables the type of data they can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data type with which it is declared. Every data type requires a different amount of memory.

C++ supports a wide variety of data types and the programmer can select the data type appropriate to the needs of the application. Data types specify the size and types of values to be stored. However, storage representation and machine instructions to manipulate each data type differ from machine to machine, although C++ instructions are identical on all machines.

C++ supports the following data types:

1. Primary or Built-in or Fundamental data type
2. Derived data types
3. User-defined data types

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **Object Oriented Programming using C++**

Unit II

Data Types in C++ are Mainly Divided into 3 Types:

1. Primitive Data Types: These data types are built-in or predefined data types and can be used directly by the user to declare variables. example: int, char, float, bool, etc. Primitive data types available in C++ are:

-  Integer
-  Character
-  Boolean
-  Floating Point
-  Double Floating Point
-  Valueless or Void
-  Wide Character

2. Derived Data Types: Derived data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely:

-  Function
-  Array
-  Pointer
-  Reference

3. Abstract or User-Defined Data Types: Abstract or User-Defined data types are defined by the user itself. Like, defining a class in C++ or a structure. C++ provides the following user-defined datatypes:

-  Class
-  Structure
-  Union
-  Enumeration
-  Typedef defined Datatype

Primitive/Basic Data Types:

- ❖ **Integer:** The keyword used for integer data types is int. Integers typically require 4 bytes of memory space and range from -2147483648 to 2147483647.
- ❖ **Character:** Character data type is used for storing characters. The keyword used for the character data type is char. Characters typically require 1 byte of memory space and range from -128 to 127 or 0 to 255.

- ❖ **Boolean:** Boolean data type is used for storing Boolean or logical values. A Boolean variable can store either true or false. The keyword used for the Boolean data type is bool.
- ❖ **Floating Point:** Floating Point data type is used for storing single-precision floating-point values or decimal values. The keyword used for the floating-point data type is float. Float variables typically require 4 bytes of memory space.
- ❖ **Double Floating Point:** Double Floating Point data type is used for storing double-precision floating-point values or decimal values. The keyword used for the double floating-point data type is double. Double variables typically require 8 bytes of memory space.
- ❖ **void:** Void means without any value. void data type represents a valueless entity. A void data type is used for those function which does not return a value.
- ❖ **Wide Character:** Wide character data type is also a character data type but this data type has a size greater than the normal 8-bit data type. Represented by wchar_t. It is generally 2 or 4 bytes long.

sizeof() operator: sizeof() operator is used to find the number of bytes occupied by a variable/data type in computer memory.

Streams in C++:

In C++ stream refers to the stream of characters that are transferred between the program thread and i/o.

Stream classes in C++ are used to input and output operations on files and io devices. These classes have specific features and to handle input and output of the program.

The iostream.h library holds all the stream classes in the C++ programming language.

1. **ios class** – This class is the base class for all stream classes. The streams can be input or output streams. This class defines members that are independent of how the templates of the class are defined.
2. **istream Class** – The istream class handles the input stream in c++ programming language. These input stream objects are used to read and interpret the input as a sequence of characters. The cin handles the input.
3. **ostream class** – The ostream class handles the output stream in c++ programming language. These output stream objects are used to write data as a sequence of characters on the screen. cout and puts handle the out streams in c++ programming language.

Operators in C++

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C++ is rich in built-in operators and provide the following types of operators –

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Misc Operators

Operators Precedence in C++

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator –

For example $x = 7 + 3 * 2$; here, x is assigned 13, not 20 because operator $*$ has higher precedence than $+$, so it first gets multiplied with $3*2$ and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **Object Oriented Programming using C++**

Unit II

Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

Manipulators in C++

The unformatted I/O statements that it is impossible to display output in a required user format or input the values in the desired form. In certain situations, we may need to format the I/O as per user requirements. For example :

- 1) The square root of a number should be displayed upto 2 decimal places.
- 2) The number to be inputted must be in a hexadecimal form.

To overcome the problems of unformatted I/O operations in C++, the concept of manipulators was introduced.

The manipulators in C++ are special functions that can be used with insertion (<<) and extraction (>>) operators to manipulate or format the data in the desired way. Certain manipulators are used with << operator to display the output in a particular format, whereas certain manipulators are used with >> operator to input the data in the desired form. The manipulators are used to set field widths, set precision, inserting a new line, skipping white space etc.

To use most of the manipulators, we need to include the header file `iomanip.h` in the program.

Manipulator	Purpose	Header File
<code>endl</code>	causes line feed to be inserted i.e. '\n'	<code>iostream.h</code>
<code>dec, oct, hex</code>	set the desired number system	<code>iostream.h</code>
<code>setbase (b)</code>	output integers in base b	<code>iomanip.h</code>

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **Object Oriented Programming using C++**

Unit II

setw(w)	read or write values to w characters	iomanip.h
setfill (c)	fills the whitespace with character c	iomanip.h
setprecision(n)	set floating point precision to n	iomanip.h

Decision Making Statement in C++

Decision making is about deciding the order of execution of statements based on certain conditions or repeat a group of statements until certain specified conditions are met. C++ handles decision-making by supporting the following statements,

1. if statement
2. switch statement

Decision making with if statement

The if statement may be implemented in different forms depending on the complexity of conditions to be tested. The different forms are-

- Simple if statement
- if....else statement
- Nested if....else statement
- else if statement

Decision making with switch statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

Syntax

The syntax for a switch statement in C++ is as follows –

```
switch(expression) {  
    case constant-expression :  
        statement(s);  
        break; //optional
```

case constant-expression :

statement(s);

break; //optional

// you can have any number of case statements.

default : //Optional

statement(s);

}

Iterative Statement (Loop)

In any programming language, loops are used to execute a set of statements repeatedly until a particular condition is satisfied.

A sequence of statement is executed until a specified condition is true. This sequence of statement to be executed is kept inside the curly braces { } known as loop body. After every execution of loop body, condition is checked, and if it is found to be true the loop body is executed again. When condition check comes out to be false, the loop body will not be executed.

There are 3 type of loops in C++ language

✚ while loop

✚ for loop

✚ do-while loop

Jump Statements

Jump statements are implemented to change the flow of the program when particular conditions are satisfied. It is used within a program to end or continue a loop or to pause the execution of a function. C++ has four jump statements: continue, break, return, and goto.

Continue:

Instead of terminating the loop, it performs the next iteration of the same loop but ignoring the parts specified by the condition. Within the loop, it must be used in conjunction with a decision-making statement. This statement can be used within a for, while, or do-while loop.

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **Object Oriented Programming using C++**

Unit II

Break:

If the condition is met, the loop is terminated with the break command. When the condition is met, the loop is broken and the remainder of the loop is skipped, unlike the continue statement. Break statements are used in conjunction with decision-making statements such as if, if-else, or switch statements in a for loop, which can be a for loop, while loop, or do-while loop. It causes the loop to stop executing future iterations.

Return:

It removes control from the function. It is more powerful than a break. It is used to end the entire function after the function has completed or after a condition has been met. Except for the void() function, every function contains a return statement that returns some value. Although the void() function can also have a return statement to conclude the function's execution.

Goto:

This statement enables us to jump directly to the section of the program that is being referenced. Each goto statement is connected with a label that directs them to the section of the programme for which they are named. Label statements can be written anywhere in the program; no before or after goto statements are required. This statement makes understanding the flow of the program difficult, hence it is avoided in program.

C++ Type compatibility

c++ is very strict with the type compatibility as compared to C language. for instance, c++ language defines int, short int, and long int as three different types, although each of these has size of one byte. In C++ language the types of values must be the same for complete compatibility, or a cast must be applied. These restrictions in c++ are necessary in order to support function overloading where two functions with the same name are distinguished using the type of function arguments

Another major difference is the way char constants are stored. in C programming, they are stored as ints, and therefore is equivalent to sizeof(int) in c language, in c++ language, however, char is not promoted to the size of int and therefore sizeof('y') equals sizeof(char)

C++ is very strict with regard to type compatibility as compared to C programming. Type compatibility is very close to implicit or automatic type conversion. The type compatibility is being able to use two types being able to substitute one for the other without modification and together without modification.

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: **Object Oriented Programming using C++**

Unit II

The type compatibility is categorized into following three types by the compiler:

1. Assignment compatibility

In assignment compatibility, if the one type of variable assigned to another type variable is different It will results into loss of value of assigned variable if the size of the assigned variable is large than the size of variable to which it is assigned.

For example

```
float f1=12.5;
```

```
int i1=f1;
```

This assigning of variable f1 float value to i1 int type will result in loss of decimal value of f1. However, this type type compatibility will not show any type error but it might give a warning “possible loss of data”.

2. Expression compatibility

Consider following example

```
int num=5/2;
```

```
cout<< num;
```

Here in the above example the result will be 2 because The actual result of 5/2 is 2.5 but because of incompatibility there will be loss of decimal value

3. Parameter compatibility

Due to incompatibility when we pass the values from calling to called function in type of actual parameter and formal parameters loss of data occurs.

For example

```
void show(int num)
```

```
{ cout << "num=" << num;
```

```
}
```

```
void main()
```

```
{ show(5.2);  
}
```

Here output will be num=5 due to incompatibility in actual and formal parameter type.

Type Conversion

There are mainly two types of type conversion. The implicit and explicit.

Implicit type conversion

This is also known as automatic type conversion. This is done by the compiler without any external trigger from the user. This is done when one expression has more than one datatype is present.

All datatypes are upgraded to the datatype of the large variable.

bool -> char -> short int -> int -> unsigned int -> long -> unsigned -> long long -> float -> double -> long double

Explicit type conversion

This is also known as type casting. Here the user can typecast the result to make it to particular datatype.

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    double x = 1.574;  
    int add = (int)x + 1;  
    cout << "Add: " << add;  
  
    float y = 3.5;  
    int val = static_cast<int>(y);  
    cout << "\nvalue: " << val;  
}
```

Functions:

A function is a block of code which only runs when it is called.

We can pass data, known as parameters, into a function.

Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.

Create/Declare a Function:

C++ provides some pre-defined functions, such as `main()`, which is used to execute code. But you can also create your own functions to perform certain actions.

To create a function, specify the name of the function, followed by parentheses ():

Syntax:

```
void myFun()  
{  
  //code to be executed  
}
```

- ***myFun()*** is the name of the function
- ***void*** means that the function does not have a return value
- inside the function (the body), add code that defines what the function should do

Call a Function

Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are called.

To call a function, write the function's name followed by two parentheses () and a semicolon ;

In the following example, `myFun()` is used to print a text (the action), when it is called:

Example -

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: Object Oriented Programming With C++

Unit III

[Functions in C++, classes & Objects]

Inside main, call myFunction():

// Create a function

```
void myFun() {  
    cout << "Hello World";  
}
```

```
int main() {  
    myFunction(); // call the function  
    return 0;  
}
```

Output : Hello World

A function can be called multiple times:

```
void myFunction() {  
    cout << endl<<" Hello World";  
}
```

```
int main() {  
    myFunction();  
    myFunction();  
    myFunction();  
    return 0;  
}
```

**Output : Hello World
Hello World
Hello World
Hello World
Hello World**

Function Declaration and Definition

A C++ function consist of two parts:

1. **Declaration:** the return type, the name of the function, and parameters (if any)
2. **Definition:** the body of the function (code to be executed)

```
void myFun() // declaration
{
    // the body of the function (definition)
}
```

Why Do We Need Functions?

- Functions help us in reducing code redundancy.
- If functionality is performed at multiple places in software, then rather than writing the same code, again and again, we create a function and call it everywhere
- This also helps in maintenance as we have to change at one place if we make future changes to the functionality.
- Functions make code modular. Consider a big file having many lines of code. It becomes really simple to read and use the code if the code is divided into functions.
- Functions provide abstraction. For example, we can use library functions without worrying about their internal work.

Types of Functions:

User Defined Function

User Defined functions are user/customer-defined blocks of code specially customized to reduce the complexity of big programs. They are also commonly known as “tailor-made functions” which are built only to satisfy the condition in which the user is facing issues meanwhile reducing the complexity of the whole program.

Library Function

Library functions are also called “builtin Functions“. These functions are a part of a compiler package that is already defined and consists of a special function with special and different

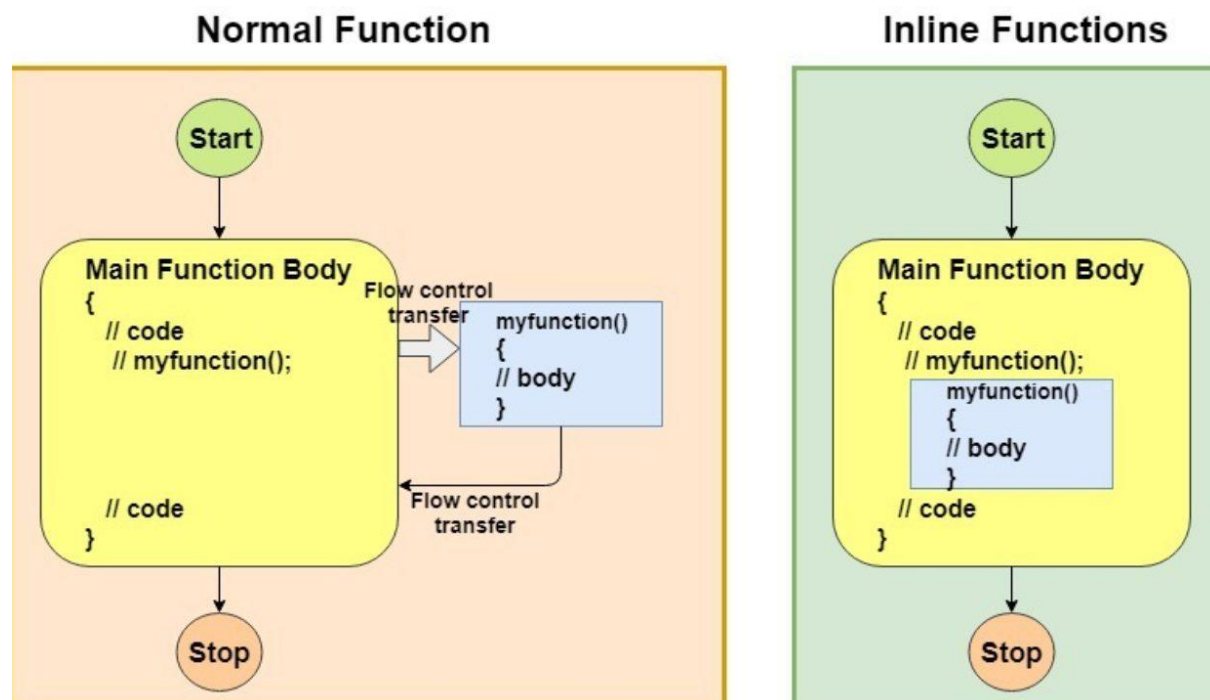
meanings. Builtin Function gives us an edge as we can directly use them without defining them whereas in the user-defined function we have to declare and define a function before using them. For Example: sqrt(), setw(), strcat(), etc.

Inline Function:

C++ provides inline functions to reduce the function call overhead. An inline function is a function that is expanded in line when it is called. When the inline function is called whole code of the inline function gets inserted or substituted at the point of the inline function call. This substitution is performed by the C++ compiler at compile time. An inline function may increase efficiency if it is small.

Syntax:

```
inline return-type function-name(parameters)
{
    // function code
}
```



Remember, inlining is only a request to the compiler, not a command. The compiler can ignore the request for inlining.

The compiler may not perform inlining in such circumstances as:

- If a function contains a loop. (for, while and do-while)
- If a function contains static variables.
- If a function is recursive.
- If a function return type is other than void, and the return statement doesn't exist in a function body.
- If a function contains a switch or goto statement.

Why Inline Functions are Used?

When the program executes the function call instruction the CPU stores the memory address of the instruction following the function call, copies the arguments of the function on the stack, and finally transfers control to the specified function. The CPU then executes the function code, stores the function return value in a predefined memory location/register, and returns control to the calling function. This can become overhead if the execution time of the function is less than the switching time from the caller function to called function (callee).

For functions that are large and/or perform complex tasks, the overhead of the function call is usually insignificant compared to the amount of time the function takes to run. However, for small, commonly-used functions, the time needed to make the function call is often a lot more than the time needed to actually execute the function's code. This overhead occurs for small functions because the execution time of a small function is less than the switching time.

Inline functions Advantages:

1. Function call overhead doesn't occur.
2. It also saves the overhead of push/pop variables on the stack when a function is called.
3. It also saves the overhead of a return call from a function.
4. When you inline a function, you may enable the compiler to perform context-specific optimization on the body of the function. Such optimizations are not possible for normal function calls. Other optimizations can be obtained by considering the flows of the calling context and the called context.
5. An inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function called preamble and return.

Inline function Disadvantages:

1. The added variables from the inlined function consume additional registers, After the inlining function if the variable number which is going to use the register increases then they may create overhead on register variable resource utilization. This means that when the inline function body is substituted at the point of the function call, the total number of variables used by the function also gets inserted. So the number of registers going to be used for the variables will also get increased. So if after function inlining variable numbers increase drastically then it would surely cause overhead on register utilization.
2. If you use too many inline functions then the size of the binary executable file will be large, because of the duplication of the same code.
3. Too much inlining can also reduce your instruction cache hit rate, thus reducing the speed of instruction fetch from that of cache memory to that of primary memory.
4. The inline function may increase compile time overhead if someone changes the code inside the inline function then all the calling location has to be recompiled because the compiler would be required to replace all the code once again to reflect the changes, otherwise it will continue with old functionality.
5. Inline functions may not be useful for many embedded systems. Because in embedded systems code size is more important than speed.
6. Inline functions might cause thrashing because inlining might increase the size of the binary executable file. Thrashing in memory causes the performance of the computer to degrade. The following program demonstrates the use of the inline function.

Example 1:

```
#include <iostream>
using namespace std;
inline int cube(int s)
{ return s * s * s; }

int main()
{
    cout << "The cube of 3 is: " << cube(3) << "\n";
    return 0;
}
```

Example 2:

```
#include <iostream>
```

```
using namespace std;
```

```
inline int Max(int x, int y) {  
    return (x > y)? x : y;  
}
```

```
// Main function for the program
```

```
int main() {  
    cout << "Max (20,10): " << Max(20,10) << endl;  
    cout << "Max (0,200): " << Max(0,200) << endl;  
    cout << "Max (100,1010): " << Max(100,1010) << endl;  
  
    return 0;  
}
```

Default arguments in C++

In a function, arguments are defined as the values passed when a function is called. Values passed are the source, and the receiving function is the destination.

A default argument is a value in the function declaration automatically assigned by the compiler if the calling function does not pass any value to that argument.

Characteristics for defining the default arguments

Following are the rules of declaring default arguments -

- The values passed in the default arguments are not constant. These values can be overwritten if the value is passed to the function. If not, the previously declared value retains.
- During the calling of function, the values are copied from left to right.
- All the values that will be given default value will be on the right.

Example

```
void function(int x, int y, int z = 0)
```

Explanation - The above function is valid. Here z is the value that is predefined as a part of the default argument.

```
Void function(int x, int z = 0, int y)
```

Explanation - The above function is invalid. Here z is the value defined in between, and it is not accepted.

```
#include<iostream>
using namespace std;
int sum(int x, int y, int z=0, int w=0) // Here there are two values in the default arguments
{ // Both z and w are initialised to zero
    return (x + y + z + w); // return sum of all parameter values
}
int main()
{
    cout << sum(10, 15) << endl; // x = 10, y = 15, z = 0, w = 0
    cout << sum(10, 15, 25) << endl; // x = 10, y = 15, z = 25, w = 0
    cout << sum(10, 15, 25, 30) << endl; // x = 10, y = 15, z = 25, w = 30
    return 0;
}
```

Function Overloading in C++

Function overloading is a feature of object-oriented programming where two or more functions can have the same name but different parameters. When a function name is overloaded with different jobs it is called Function Overloading. In Function Overloading “Function” name should be the same and the arguments should be different. Function overloading can be considered as an example of a polymorphism feature in C++.

If multiple functions having same name but parameters of the functions should be different is known as Function Overloading.

If we have to perform only one operation and having same name of the functions increases the readability of the program.

Suppose you have to perform addition of the given numbers but there can be any number of arguments, if you write the function such as `a(int,int)` for two parameters, and `b(int,int,int)` for three parameters then it may be difficult for you to understand the behavior of the function because its name differs.

The parameters should follow any one or more than one of the following conditions for Function overloading:

1) Parameters should have a different type

`add(int a, int b)`

`add(double a, double b)`

2) Parameters should have a different number

`add(int a, int b)`

`add(int a, int b, int c)`

3) Parameters should have a different sequence of parameters.

`add(int a, double b)`

`add(double a, int b)`

Example 1 : Parameters should have a different type

```
#include <iostream>
```

```
using namespace std;
```

```
void add(int a, int b)
```

```
{ cout << "sum = " << (a + b);
```

```
}
```

```
void add(double a, double b)
```

```
{ cout << endl << "sum = " << (a + b);
```

```
}
```

```
int main()
```

```
{    add(10, 2);
```

```
    add(5.3, 6.2);
```

```
    return 0;
```

```
}
```


Example 2 : Parameters should have a different number

```
#include <iostream>
using namespace std;

void add(int a, int b)
{cout << "sum = " << (a + b);
}

void add(int a, int b, int c)
{cout << endl << "sum = " << (a + b + c);
}

int main()
{    add(10, 2);
    add(5, 6, 4);
    return 0;
}
```

Example 3 : Parameters should have a different sequence of parameters.

```
#include<iostream>
using namespace std;

void add(int a, double b)
{cout<<"sum = "<<(a+b);
}

void add(double a, int b)
{cout<<endl<<"sum = "<<(a+b);
}

int main()
{    add(10,2.5);
    add(5.5,6);
    return 0;
}
```

Call by Value and Call by Reference:

Functions can be invoked in two ways: Call by Value or Call by Reference. These two ways are generally differentiated by the type of values passed to them as parameters.

The parameters passed to the function are called actual parameters whereas the parameters received by the function are called formal parameters.

Call By Value:

In call by value method of parameter passing, the values of actual parameters are copied to the function's formal parameters.

1. There are two copies of parameters stored in different memory locations.
2. One is the original copy and the other is the function copy.
3. Any changes made inside functions are not reflected in the actual parameters of the caller.

The following example demonstrates the call-by-value method of parameter passing

```
#include <stdio.h>
void swapx(int x, int y);

int main()
{
    int a = 10, b = 20;
    // Pass by Values
    swapx(a, b);
    printf("In the Caller:\na = %d b = %d\n", a, b);
    return 0;
}

void swapx(int x, int y)
{
    int t;
    t = x;
    x = y;
    y = t;
    printf("Inside Function:\nx = %d y = %d\n", x, y);
}
```

Call by Reference

In call by reference method of parameter passing, the address of the actual parameters is passed to the function as the formal parameters.

1. Both the actual and formal parameters refer to the same locations.
2. Any changes made inside the function are actually reflected in the actual parameters of the caller.

The following C program is an example of call by reference method.

```
#include <stdio.h>
```

```
void swapx(int*, int*);
```

```
int main()
```

```
{
```

```
    int a = 10, b = 20;
```

```
    // Pass reference
```

```
    swapx(&a, &b);
```

```
    printf("Inside the Caller:\na = %d b = %d\n", a, b);
```

```
    return 0;
```

```
}
```

```
void swapx(int* x, int* y)
```

```
{
```

```
    int t;
```

```
    t = *x;
```

```
    *x = *y;
```

```
    *y = t;
```

```
    printf("Inside the Function:\nx = %d y = %d\n", *x, *y);
```

```
}
```

Classes:

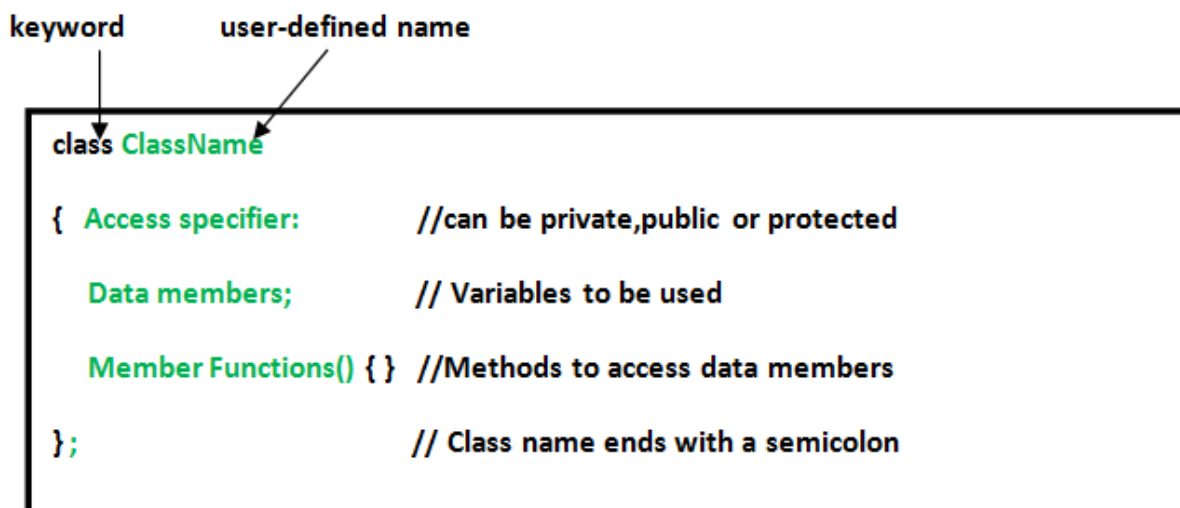
Class in C++ is the building block that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object. For Example: Consider the Class of Cars. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc. So here, Car is the class, and wheels, speed limits, and mileage are their properties.

- A Class is a user-defined data type that has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables together, these data members and member functions define the properties and behavior of the objects in a Class.
- In the above example of class Car, the data member will be speed limit, mileage, etc, and member functions can be applying brakes, increasing speed, etc.

An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using the keyword class followed by the name of the class. The body of the class is defined inside the curly brackets and terminated by a semicolon at the end.



Declaring Objects

When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

ClassName ObjectName;

Accessing data members and member functions: The data members and member functions of the class can be accessed using the dot('.') operator with the object. For example, if the name of the object is obj and you want to access the member function with the name printName() then you will have to write obj.printName().

Array of Objects in C++

Like array of other user-defined data types, an array of type class can also be created. The array of type class contains the objects of the class as its individual elements. Thus, an array of a class type is also known as an array of objects. An array of objects is declared in the same way as an array of any built-in data type.

The syntax for declaring an array of objects is

class_name array_name [size] ;

To understand the concept of an array of objects, consider this example.

Example : A program to demonstrate the concept of array of objects

```
#include<iostream>
using namespace std;
class books {
    char title [30];
    float price ;
public:
    void getdata ();
    void putdata ();
};
void books :: getdata () {
    cout<<"Title:";
    cin>>title;
```

```
    cout<<"Price:";
    cin>>price;
}
void books :: putdata () {
    cout<<"Title:"<<title<< "\n";
    cout<<"Price:"<<price<< "\n";
    const int size=3 ;
}
int main() {
    books book[size] ;
    for(int i=0;i<size;i++) {
        cout<<"Enter details of book "<<(i+1)<<"\n";
        book[i].getdata();
    }
    for(int i=0;i<size;i++) {
        cout<<"\nBook "<<(i+1)<<"\n";
        book[i].putdata() ;
    }
    return 0;
}
```

Passing and Returning Objects in C++:

In C++ we can pass class's objects as arguments and also return them from a function the same way we pass and return other variables. No special keyword or header file is required to do so.

Passing an Object as argument

To pass an object as an argument we write the object name as the argument while calling the function the same way we do it for other variables.

Syntax:

```
function_name(object_name);
```

// C++ program to show passing of objects to a function

```
class Example {
public:
    int a;

    // This function will take an object as an argument
    void add(Example E)
    {
        a = a + E.a;
    }
};

int main()
{
    Example E1, E2;

    E1.a = 50;
    E2.a = 100;

    cout << "Initial Values \n";
    cout << "Value of object 1: " << E1.a
        << "\n& object 2: " << E2.a
        << "\n\n";

    // Passing object as an argument to function add()
    E2.add(E1);

    // Changed values after passing object as argument
    cout << "New values \n";
    cout << "Value of object 1: " << E1.a
        << "\n& object 2: " << E2.a
        << "\n\n";

    return 0;
}
```

Static Member Function in C++

The static is a keyword in the C and C++ programming language. We use the static keyword to define the static data member or static member function inside and outside of the class. Let's understand the static data member and static member function using the programs.

Static data member

When we define the data member of a class using the static keyword, the data members are called the static data member. A static data member is similar to the static member function because the static data can only be accessed using the static data member or static member function. And, all the objects of the class share the same copy of the static member to access the static data.

Syntax

static data_type data_member;

Here, the static is a keyword of the predefined library.

The data_type is the variable type in C++, such as int, float, string, etc.

The data_member is the name of the static data.

Example: Let's create a simple program to access the static data members in the C++ programming language.

```
#include <iostream>
#include <string.h>
using namespace std;

class Car
{
private:
int car_id;
char car_name[20];
int marks;

public:
// declare a static data member
static int static_member;
```



```
Car()
{
static_member++;
}

void inp()
{
cout << " \n\n Enter the Id of the Car: " << endl;
cin >> car_id; // input the id
cout << " Enter the name of the Car: " << endl;
cin >> car_name;
cout << " Number of the Marks (1 - 10): " << endl;
cin >> marks;
}

void disp ()
{
cout << " \n Id of the Car: " << car_id;
cout << " \n Name of the Car: " << car_name;
cout << " \n Marks: " << marks;

}
};

// initialized the static data member to 0
int Car::static_member = 0;

int main ()
{
// create object for the class Car
Car c1;
// call inp() function to insert values
c1. inp ();
c1. disp();

//create another object
Car c2;
```

```
// call inp() function to insert values
c2. inp ();
c2. disp();
cout << " \n No. of objects created in the class: " << Car :: static_member <<endl;
return 0;
}
```

Static Member Functions

The static member functions are special functions used to access the static data members or other static member functions. A member function is defined using the static keyword. A static member function shares the single copy of the member function to any number of the class' objects. We can access the static member function using the class name or class' objects. If the static member function accesses any non-static data member or non-static member function, it throws an error.

Syntax

class_name::function_name (parameter);

Here, the class_name is the name of the class.

function_name: The function name is the name of the static member function.

parameter: It defines the name of the pass arguments to the static member function.

Example: Let's create another program to access the static member function using the class name in the C++ programming language.

```
#include <iostream>
using namespace std;
class Note
{
// declare a static data member
static int num;

public:
// create static member function
```

```
static int func ()
{
    return num;
}
};
// initialize the static data member using the class name and the scope resolution operator
int Note :: num = 5;

int main ()
{
    // access static member function using the class name and the scope resolution
    cout << " The value of the num is: " << Note:: func () << endl;
    return 0;
}
```

Output

The value of the num is: 5

C++ Friend function

If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.

By using the keyword friend compiler knows the given function is a friend function.

For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function in C++

```
class class_name
{
    friend data_type function_name(arguments);
};
```

[Functions in C++, classes & Objects]

In the above declaration, the friend function is preceded by the keyword friend. The function can be defined anywhere in the program like a normal C++ function. The function definition does not use either the keyword friend or scope resolution operator.

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.
- C++ friend function Example

Simple example of C++ friend function used to print the length of a box.

```
#include <iostream>
using namespace std;
class Box
{
    private:
        int length;
    public:
        Box(): length(0) { }
        friend int printLength(Box); //friend function
};
int printLength(Box b)
{
    b.length += 10;
    return b.length;
}
int main()
{
    Box b;
    cout<<"Length of box: "<< printLength(b)<<endl;
    return 0;
}
```

Output:

Length of box: 10

Constructor

A constructor is a special member function of a class and shares the same name as of class, which means the constructor and class have the same name. Constructor is called by the compiler whenever the object of the class is created, it allocates the memory to the object and initializes class data members by default values or values passed by the user while creating an object. Constructors don't have any return type because their work is to just create and initialize an object.

The basic syntax of the constructor is given below:

```
class class_name{
    private:
        // private members

    public:

        // declaring constructor
        class_name({parameters})
        {
            // constructor body
        }
};
```

Characteristics of constructor:

- The name of the constructor is same as its class name.
- Constructors are mostly declared in the public section of the class though it can be declared in the private section of the class.
- Constructors do not return values; hence they do not have a return type.
- A constructor gets called automatically when we create the object of the class.
- Constructors can be overloaded.
- Constructor can not be declared virtual.

Types of constructors in C++:

- Default constructor
- Parameterized constructor
- Copy Constructor

Default constructor :

Default constructor is the constructor which doesn't take any argument. It has no parameters. It is also called a zero-argument constructor.

```
#include <iostream>
using namespace std;
```

```
class construct
```

```
{
```

```
public:
```

```
    int a, b;
```

```
    // Default Constructor
```

```
    construct()
```

```
    {    a = 10;
```

```
        b = 20;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    // Default constructor called automatically
```

```
    // when the object is created
```

```
    construct c;
```

```
    cout << "a: " << c.a << endl << "b: " << c.b;
```

```
    return 1;
```

```
}
```

[Constructor & Destructor, Operator overloading, Inheritance]

Parameterized Constructors:

It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

```
#include <iostream>
using namespace std;

class Point {
private:
    int x, y;

public:
    // Parameterized Constructor
    Point(int x1, int y1)
    {
        x = x1;
        y = y1;
    }

    int getX() { return x; }
    int getY() { return y; }
};

int main()
{
    // Constructor called
    Point p1(10, 15);

    // Access values assigned by constructor
    cout << "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();
    return 0;
}
```

[Constructor & Destructor, Operator overloading, Inheritance]

Copy Constructor:

A copy constructor is a member function that initializes an object using another object of the same class. A detailed article on Copy Constructor.

Whenever we define one or more non-default constructors(with parameters) for a class, a default constructor(without parameters) should also be explicitly defined as the compiler will not provide a default constructor in this case. However, it is not necessary but it's considered to be the best practice to always define a default constructor.

Copy constructor takes a reference to an object of the same class as an argument.

```
Sample(Sample &t)
{
    id=t.id;
}
```

```
#include<iostream>
using namespace std;
```

```
class Sample
{
    int id;
public:
    void init(int x)
    { id=x;
    }
    void display()
    {cout<<endl<<"ID="<<id;
    }
};
```

```
int main()
{
    Sample obj1;
    obj1.init(10);
    obj1.display();

    Sample obj2(obj1); //or obj2=obj1;
    obj2.display();
    return 0;
}
```

Destructor

A destructor works opposite to constructor; it destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

A destructor is defined like constructor. It must have same name as class. But it is prefixed with a tilde sign (~).

```
#include <iostream>
using namespace std;
class Employee
{
public:
    Employee()
    {
        cout<<"Constructor Invoked"<<endl;
    }
    ~Employee()
    {
        cout<<"Destructor Invoked"<<endl;
    }
};
int main(void)
{
    Employee e1; //creating an object of Employee
    Employee e2; //creating an object of Employee
    return 0;
}
```

Operator Overloading

Operator overloading is a compile-time polymorphism. It is an idea of giving special meaning to an existing operator in C++ without changing its original meaning.

In C++, we can make operators work for user-defined classes. This means C++ has the ability to provide the operators with a special meaning for a data type, this ability is known as operator overloading. For example, we can overload an operator '+' in a class like String so that we can concatenate two strings by just using +. Other example classes where arithmetic operators may be overloaded are Complex Numbers, Fractional Numbers, Big integers, etc.

The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

1. Scope operator (::)
2. Sizeof
3. member selector(.)
4. member pointer selector(*)
5. ternary operator(?:)

Syntax of Operator Overloading

```
return_type class_name :: operator op(argument_list)
```

```
{
```

```
    // body of the function.
```

```
}
```

Where the return type is the type of value returned by the function.

class_name is the name of the class.

operator op is an operator function where op is the operator being overloaded, and the operator is the keyword.

Rules for Operator Overloading

1. Existing operators can only be overloaded, but the new operators cannot be overloaded.
2. The overloaded operator contains atleast one operand of the user-defined data type.
3. We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.
4. When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
5. When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

1. Overloading Unary Operator

Let us consider overloading (-) unary operator. In the unary operator function, no arguments should be passed. It works only with one class object. It is the overloading of an operator operating on a single operand.

Example: Assume that class Distance takes two member objects i.e. feet and inches, and creates a function by which the Distance object should decrement the value of feet and inches by 1 (having a single operand of Distance Type).

// C++ program to show unary operator overloading

```
#include <iostream>
```

```
using namespace std;
```

```
class Distance {
```

```
public:
```

```
    int feet, inch;
```

```
    Distance(int f, int i)
```

```
    {
```

```
        this->feet = f;
```

```
        this->inch = i;
```

```
    }
```

[Constructor & Destructor, Operator overloading, Inheritance]

```
// Overloading(-) operator to perform decrement operation of Distance object
void operator-()
{
    feet--;
    inch--;
    cout << "\nFeet & Inches(Decrement): " <<
        feet << "" << inch;
}
};

int main()
{
    Distance d1(8, 9);

    // Use (-) unary operator by single operand
    -d1;
    return 0;
}
```

2. Overloading Binary Operator

In the binary operator overloading function, there should be one argument to be passed. It is the overloading of an operator operating on two operands. Below is the C++ program to show the overloading of the binary operator (+) using a class Distance with two distant objects.

```
// C++ program to show binary operator overloading
#include <iostream>
using namespace std;

class Distance {
public:
    int feet, inch;

    Distance()
    {
        this->feet = 0;
    }
};
```

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: Object Oriented Programming With C++

Unit IV

[Constructor & Destructor, Operator overloading, Inheritance]

```
        this->inch = 0;
    }

    Distance(int f, int i)
    {
        this->feet = f;
        this->inch = i;
    }

    // Overloading (+) operator to perform addition of two distance object Call by reference
    Distance operator+(Distance& d2)
    {
        // Create an object to return
        Distance d3;

        d3.feet = this->feet + d2.feet;
        d3.inch = this->inch + d2.inch;

        // Return the resulting object
        return d3;
    }
};

int main()
{
    Distance d1(8, 9);
    Distance d2(10, 2);
    Distance d3;

    // Use overloaded operator
    d3 = d1 + d2;

    cout << "\nTotal Feet & Inches: " << d3.feet << " " << d3.inch;
    return 0;
}
```

Inheritance

In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.

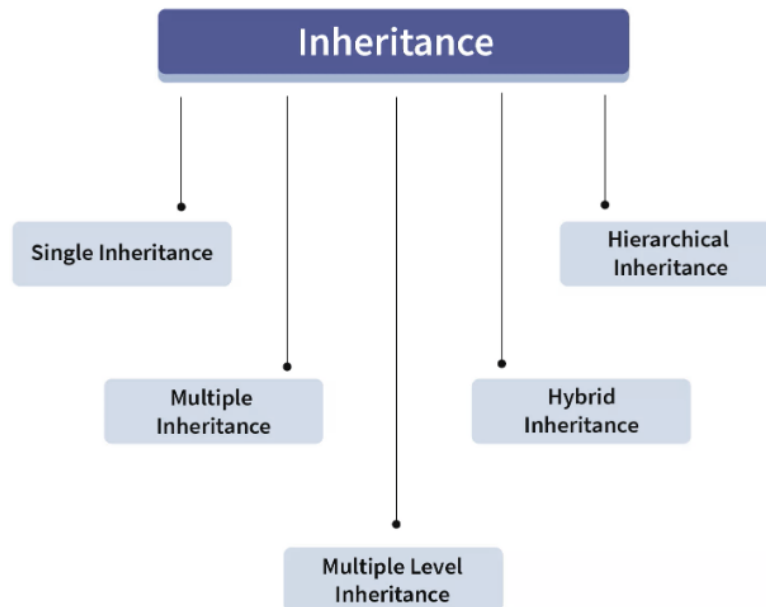
In C++, the class which inherits the members of another class is called derived class and the class whose members are inherited is called base class. The derived class is the specialized class for the base class.

Advantage of C++ Inheritance

Code reusability: Now you can reuse the members of your parent class. So, there is no need to define the member again. So less code is required in the class.

Types Of Inheritance

- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance
- Hybrid inheritance



Single Inheritance:

Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class. Where 'A' is the base class, and 'B' is the derived class.

Multilevel Inheritance

Multilevel inheritance is a process of deriving a class from another derived class.

Multiple Inheritance

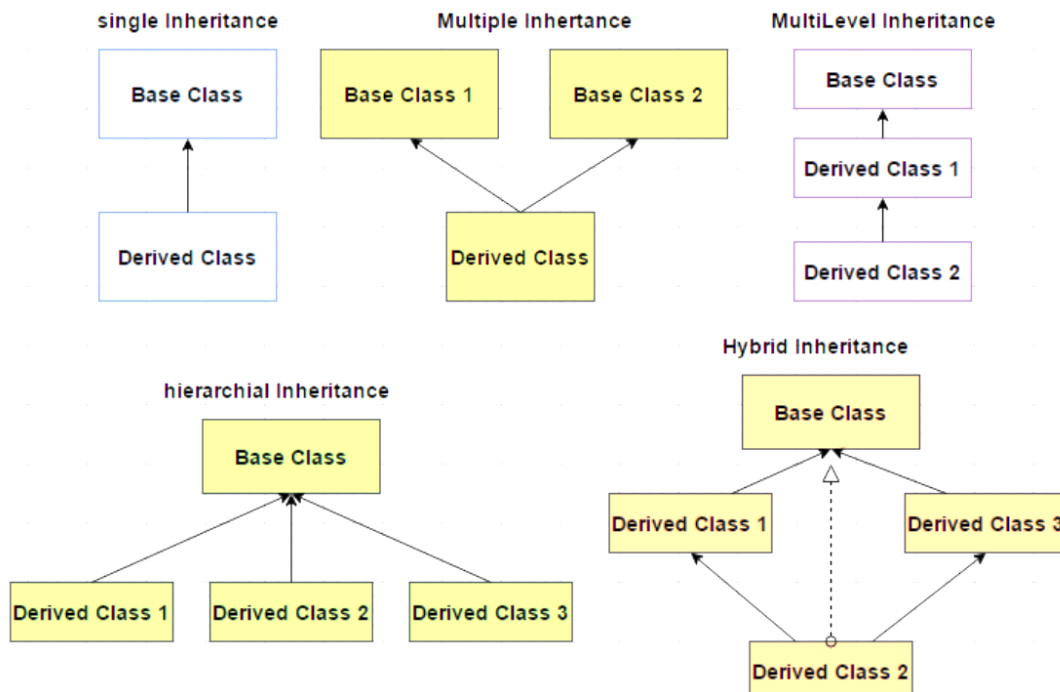
Multiple inheritance is the process of deriving a new class that inherits the attributes from two or more classes.

Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.

Hierarchical Inheritance

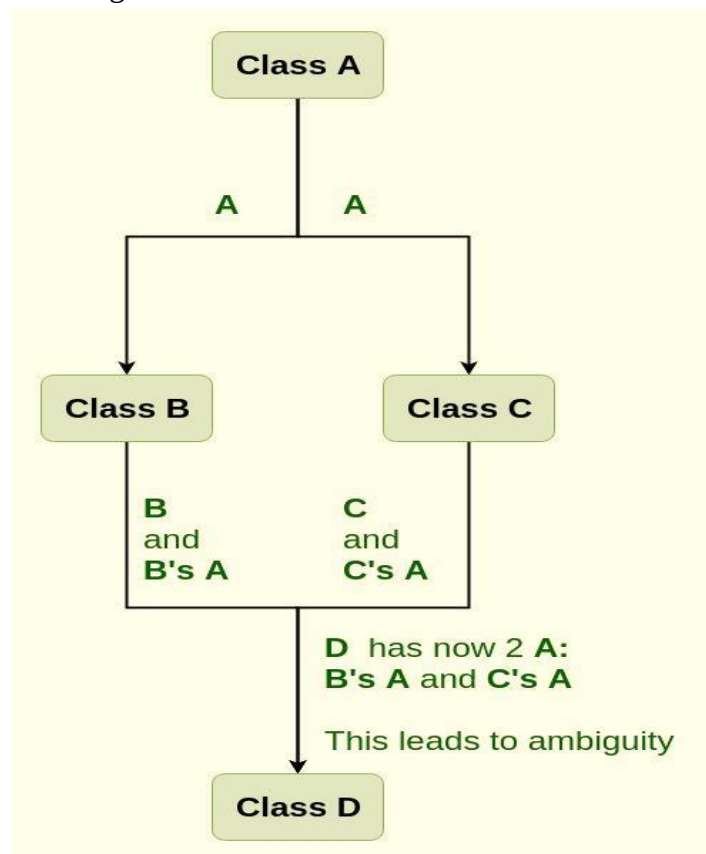
Hierarchical inheritance is defined as the process of deriving more than one class from a base class.



Virtual Base Class:

Virtual base classes are used in virtual inheritance in a way of preventing multiple “instances” of a given class appearing in an inheritance hierarchy when using multiple inheritances.

Need for Virtual Base Classes: Consider the situation where we have one class A . This class A is inherited by two other classes B and C. Both these class are inherited into another in a new class D as shown in figure below.



As we can see from the figure that data members/function of class A are inherited twice to class D. One through class B and second through class C. When any data / function member of class A is accessed by an object of class D, ambiguity arises as to which data/function member would be called? One inherited through B or the other inherited through C. This confuses compiler and it displays error.

```
#include <iostream>
```


[Constructor & Destructor, Operator overloading, Inheritance]

```
using namespace std;
```

```
class A {  
public:  
    void show()  
    {cout << "Hello form A \n";  
    }  
};
```

```
class B : public A {  
};
```

```
class C : public A {  
};
```

```
class D : public B, public C {  
};
```

```
int main()  
{    D object;  
    object.show();  
}
```

virtual can be written before or after the public. Now only one copy of data/function member will be copied to class C and class B and class A becomes the virtual base class. Virtual base classes offer a way to save space and avoid ambiguities in class hierarchies that use multiple inheritances. When a base class is specified as a virtual base, it can act as an indirect base more than once without duplication of its data members. A single copy of its data members is shared by all the base classes that use virtual base.

```
#include <iostream>  
using namespace std;
```

```
class A {  
public:  
    int a;  
    A() // constructor  
    {a = 10;
```

```
    }  
};  
  
class B : public virtual A {  
};  
  
class C : public virtual A {  
};  
  
class D : public B, public C {  
};  
  
int main()  
{   D object;  
    cout << "a = " << object.a << endl;  
    return 0;  
}
```

Abstract Class & Pure Virtual Function:

Sometimes implementation of all function cannot be provided in a base class because we don't know the implementation. Such a class is called abstract class. For example, let Shape be a base class. We cannot provide implementation of function draw() in Shape, but we know every derived class must have implementation of draw(). Similarly an Animal class doesn't have implementation of move() (assuming that all animals move), but all animals must know how to move. We cannot create objects of abstract classes.

A pure virtual function (or abstract function) in C++ is a virtual function for which we can have implementation, But we must override that function in the derived class, otherwise the derived class will also become abstract class.

```
// An abstract class  
class Test  
{  
    // Data members of class  
public:  
    // Pure Virtual Function  
    virtual void show() = 0;
```

```
/* Other members */  
};
```

A pure virtual function is implemented by classes which are derived from a Abstract class. Following is a simple example.

```
#include<iostream>  
using namespace std;  
  
class Base  
{  
int x;  
public:  
    virtual void fun() = 0;  
    int getX() { return x; }  
};  
  
// This class inherits from Base and implements fun()  
class Derived: public Base  
{  
    int y;  
public:  
    void fun() { cout << "fun() called"; }  
};  
  
int main(void)  
{  
    Derived d;  
    d.fun();  
    return 0;  
}
```

Pointer:

The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.

A pointer however, is a variable that stores the memory address as its value. Pointers are symbolic representations of addresses.

The symbol of an address is represented by a pointer. In addition to creating and modifying dynamic data structures, they allow programs to emulate call-by-reference.

Syntax:

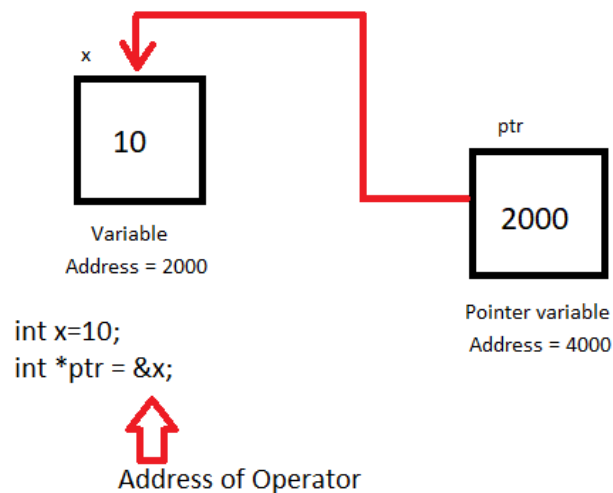
`datatype *var_name;`

`int *ptr; // ptr can point to an address which holds int data`

How to use a pointer?

- Define a pointer variable
- Assigning the address of a variable to a pointer using the unary operator (&) which returns the address of that variable.
- Accessing the value stored in the address using unary operator (*) which returns the value of the variable located at the address specified by its operand.

Since the data type knows how many bytes the information is held in, we associate it with a reference. The size of the data type to which a pointer points is added when we increment a pointer.



Advantage of pointer:

- 1) Pointer reduces the code and improves the performance, it is used to retrieving strings, trees etc. and used with arrays, structures and functions.
- 2) We can return multiple values from function using pointer.
- 3) It makes you able to access any memory location in the computer's memory.

Usage of pointer:

There are many usage of pointers in C++ language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using malloc() and calloc() functions where pointer is used.

2) Arrays, Functions and Structures

Pointers in c language are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
* (asterisk sign)	Indirection operator	Access the value of an address.

Pointer Example

```
#include <iostream>
using namespace std;
int main()
{
    int number=20;
    int *p;
    p=&number;    //stores the address of number variable
    cout<<"Address of number variable is:"<<&number<<endl;
    cout<<"Address of p variable is:"<<p<<endl;
    cout<<"Value of p variable is:"<<*p<<endl;
    return 0;
}
```

Output:

Address of number variable is:0x7ffccc8724c5

Address of p variable is:0x7ffccc8724c5

Value of p variable is: 20

Pointer to Object:

A pointer to an object acts the same as Pointer to a variable. But here, in place of the address of the variable, address of the object is stored.

In the main function, when an object is created to a class, a pointer variable is declared in the same manner as we declared for the variable, and it will store the object's address.

For creating a pointer to an object, we should not use data type for the Pointer. Instead, we need to use the class name for the object pointer. If we want to use a member function in the class using the Pointer in the main function, then we need to use the -> symbol

```
class Rectangle
{
    private:
        int length;
        int breadth;
    public:
        Rectangle(int l, int b)
        {
            length=l;
            breadth=b;
        }
        int getArea()
        { return (length*breadth);
        }
};
```

```
int main()
{
    // creating an object of Rectangle
    Rectangle R(10,30);

    //creating a pointer for the object using class name as data type
    Rectangle *ptr = &R;
```

```
//calling the member function using -> symbol
int area = ptr->getArea();
cout<<"Area of rectangle is: "<<area;
return 0;
}
```

this pointer:

In C++ programming, this is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.

- It can be used to pass current object as a parameter to another method.
- It can be used to refer current class instance variable.
- It can be used to declare indexers.

```
#include <iostream>
using namespace std;
class Employee {
public:
    int id; //data member (also instance variable)
    string name; //data member(also instance variable)
    float salary;
    Employee(int id, string name, float salary)
    {
        this->id = id;
        this->name = name;
        this->salary = salary;
    }
    void display()
    {
        cout<<id<<" "<<name<<" "<<salary<<endl;
    }
};

int main(void) {
    Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of Employee
    Employee e2=Employee(102, "Nakul", 59000); //creating an object of Employee
    e1.display();
}
```

```
e2.display();  
return 0;  
}
```

Pointer to Derive Class:

A pointer is a data type that stores the address of other data types. Pointers can be used for base objects as well as objects of derived classes. A pointer to the object of the derived class and a pointer to the object of the base class are type-compatible.

```
B *cptr; // here is pointer to class B type variable  
B b; // base object  
D d; // derived object  
cptr=&b; // cptr points to object b
```

Virtual Function

1. A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
2. It is used to tell the compiler to perform dynamic linkage or late binding on the function.
3. There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.
4. A 'virtual' is a keyword preceding the normal declaration of a function.
5. When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.
6. Late binding or Dynamic linkage
7. In late binding function call is resolved during runtime. Therefore compiler determines the type of object at runtime, and then binds the function call.

Rules of Virtual Function

1. Virtual functions must be members of some class.
2. Virtual functions cannot be static members.
3. They are accessed through object pointers.
4. They can be a friend of another class.
5. A virtual function must be defined in the base class, even though it is not used.

6. The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
7. We cannot have a virtual constructor, but we can have a virtual destructor
8. Consider the situation when we don't use the virtual keyword.

```
#include <iostream>
using namespace std;
class A
{
    int x=5;
    public:
    void display()
    {
        std::cout << "Value of x is : " << x<<std::endl;
    }
};
class B: public A
{
    int y = 10;
    public:
    void display()
    {
        std::cout << "Value of y is : " <<y<< std::endl;
    }
};
int main()
{
    A *a;
    B b;
    a = &b;
    a->display();
    return 0;
}
```

Output:

Value of x is : 5

Pure Virtual Function:

1. A virtual function is not used for performing any task. It only serves as a placeholder.
2. When the function has no definition, such function is known as "do-nothing" function.
3. The "do-nothing" function is known as a pure virtual function. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
4. A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
5. The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.

Pure virtual function can be defined as:

```
virtual void display() = 0;
```

Example:

```
#include <iostream>
using namespace std;
class Base
{ public:
    virtual void show() = 0;
};
class Derived : public Base
{
    public:
    void show()
    { cout << "Derived class is derived from the base class." << endl;
    }
};
int main()
{
    Base *bptr;
    //Base b;
    Derived d;
    bptr = &d;
    bptr->show();
    return 0;
}
```

Output:

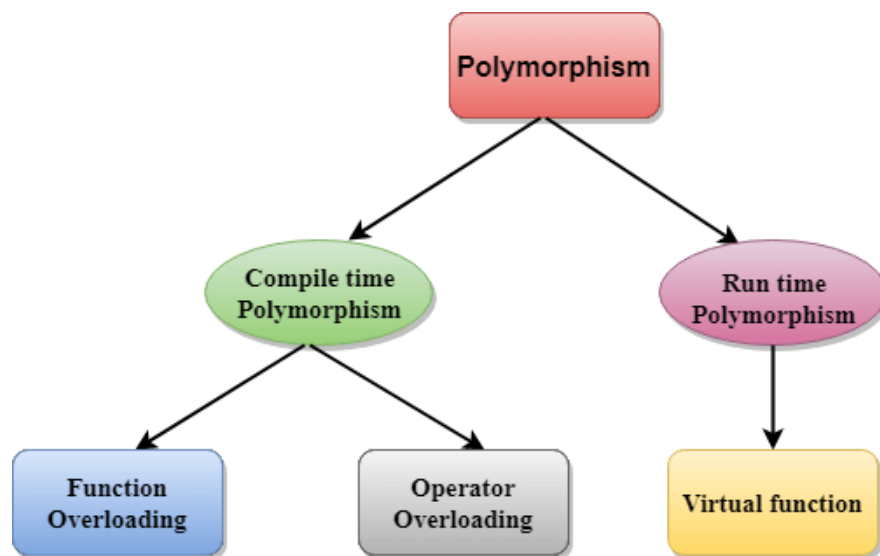
Derived class is derived from the base class.

Polymorphism

The word “polymorphism” means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. A real-life example of polymorphism is a person who at the same time can have different characteristics. A man at the same time is a father, a husband, and an employee. So the same person exhibits different behavior in different situations. This is called polymorphism. Polymorphism is considered one of the important features of Object-Oriented Programming.

Types of Polymorphism

- Compile-time Polymorphism
- Runtime Polymorphism



Compile time polymorphism: The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding. Now, let's consider the case where function name and prototype is same.

Run time polymorphism: Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

RENAISSANCE UNIVERSITY, INDORE

BCA II SEM

Subject: Object Oriented Programming With C++

Unit V

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

Function overloading:

Function overloading is a feature of object-oriented programming where two or more functions can have the same name but different parameters. When a function name is overloaded with different jobs it is called Function Overloading. In Function Overloading “Function” name should be the same and the arguments should be different. Function overloading can be considered as an example of a polymorphism feature in C++.

If multiple functions having same name but parameters of the functions should be different is known as Function Overloading.

If we have to perform only one operation and having same name of the functions increases the readability of the program.

Suppose you have to perform addition of the given numbers but there can be any number of arguments, if you write the function such as a(int,int) for two parameters, and b(int,int,int) for three parameters then it may be difficult for you to understand the behavior of the function because its name differs.

The parameters should follow any one or more than one of the following conditions for Function overloading:

Parameters should have a different type

add(int a, int b)

add(double a, double b)

```
#include <iostream>
```

```
using namespace std;
```

```
void add(int a, int b)
```

```
{ cout << "sum = " << (a + b);
```

```
}
```

```
void add(double a, double b)
```

```
{ cout << endl << "sum = " << (a + b);
```

```
}
```

```
int main()
```

```
{
```

```
    add(10, 2);
```

```
    add(5.3, 6.2);
```

```
    return 0;
```

```
}
```

Function Overriding:

If derived class defines same function as defined in its base class, it is known as function overriding in C++. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the function which is already provided by its base class.

```
#include <iostream>
```

```
using namespace std;
```

```
class Animal
```

```
{ public:
```

```
    void eat()
```

```
    { cout<<"Eating...";
```

```
    }
```

```
};
```

```
class Dog: public Animal
{ public:
    void eat()
    { cout<<"Eating bread...";
    }
};

int main(void) {
    Dog d = Dog();
    d.eat();
    return 0;
}
```

Output:

Eating bread..

Exception Handling

When executing C++ code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.

When an error occurs, C++ will normally stop and generate an error message. The technical term for this is: C++ will throw an exception (throw an error).

One of the advantages of C++ over C is Exception Handling. Exceptions are runtime anomalies or abnormal conditions that a program encounters during its execution. which are beyond the program's control, such as disc failure, keyboard interrupts etc.).

C++ provides the following specialized keywords for this purpose:

try: Represents a block of code that can throw an exception.

catch: Represents a block of code that is executed when a particular exception is thrown.

throw: Used to throw an exception. Also used to list the exceptions that a function throws but doesn't handle itself.

```
try {
    // Block of code to try
    throw exception; // Throw an exception when a problem arise
}
catch () {
    // Block of code to handle errors
}
```

Example:

```
try {
    int age = 15;
    if (age >= 18) {
        cout << "Access granted - you are old enough.";
    } else {
        throw (age);
    }
}
catch (int myNum) {
    cout << "Access denied - You must be at least 18 years old.\n";
    cout << "Age is: " << myNum;
}
```

Template

A C++ template is a powerful feature added to C++. It allows you to define the generic classes and generic functions and thus provides support for generic programming. Generic programming is a technique where generic types are used as parameters in algorithms so that they can work for a variety of data types.

Templates can be represented in two ways:

1. Function templates
2. Class templates

Function Templates:

We can define a template for a function. For example, if we have an add() function, we can create versions of the add function for adding the int, float or double type values.

Class Template:

We can define a template for a class. For example, a class template can be created for the array class that can accept the array of various types such as int array, float array or double array.

Function Template

Generic functions use the concept of a function template. Generic functions define a set of operations that can be applied to the various types of data.

The type of the data that the function will operate on depends on the type of the data passed as a parameter.

For example, Quick sorting algorithm is implemented using a generic function, it can be implemented to an array of integers or array of floats.

A Generic function is created by using the keyword template. The template defines what function will do.

Syntax of Function Template

```
template < class Ttype> ret_type func_name(parameter_list)
{
    // body of function.
}
```

Where Ttype: It is a placeholder name for a data type used by the function. It is used within the function definition. It is only a placeholder that the compiler will automatically replace this placeholder with the actual data type.

class: A class keyword is used to specify a generic type in a template declaration.

```
#include <iostream>
using namespace std;
template<class T> T add(T &a,T &b)
{
    T result = a+b;
    return result;
}
int main()
{
    int i =2;
    int j =3;
    float m = 2.3;
    float n = 1.2;
    cout<<"Addition of i and j is :"<<add(i,j);
    cout<<"\n";
    cout<<"Addition of m and n is :"<<add(m,n);
    return 0;
}
```


Class templates:

Class templates like function templates, class templates are useful when a class defines something that is independent of the data type. Can be useful for classes like LinkedList, BinaryTree, Stack, Queue, Array, etc.

```
// C++ Program to implement template Array class
```

```
#include <iostream>
```

```
using namespace std;
```

```
template <typename T> class Array {
```

```
private:
```

```
    T* ptr;
```

```
    int size;
```

```
public:
```

```
    Array(T arr[], int s);
```

```
    void print();
```

```
};
```

```
template <typename T> Array<T>::Array(T arr[], int s)
```

```
{
```

```
    ptr = new T[s];
```

```
    size = s;
```

```
    for (int i = 0; i < size; i++)
```

```
        ptr[i] = arr[i];
```

```
}
```

```
template <typename T> void Array<T>::print()
```

```
{
```

```
    for (int i = 0; i < size; i++)
```

```
        cout << " " << *(ptr + i);
```

```
    cout << endl;
```

```
}
```

```
int main()
```

```
{
```

```
    int arr[5] = { 1, 2, 3, 4, 5 };
```

```
    Array<int> a(arr, 5);  
    a.print();  
    return 0;  
}
```

File Handling

File handling is used to store data permanently in a computer. Using file handling we can store our data in secondary memory (Hard disk).

How to achieve the File Handling

For achieving file handling we need to follow the following steps:-

STEP 1-Naming a file

STEP 2-Opening a file

STEP 3-Writing data into the file

STEP 4-Reading data from the file

STEP 5-Closing a file.

Streams in C++ :-

We give input to the executing program and the execution program gives back the output. The sequence of bytes given as input to the executing program and the sequence of bytes that comes as output from the executing program are called stream. In other words, streams are nothing but the flow of data in a sequence.

The input and output operation between the executing program and the devices like keyboard and monitor are known as “console I/O operation”. The input and output operation between the executing program and files are known as “disk I/O operation”.

1. **ofstream:** This data type represents the output file stream and is used to create files and to write information to files.
2. **ifstream:** This data type represents the input file stream and is used to read information from files.
3. **fstream:** This data type represents the file stream generally, and has the capabilities of both ofstream and ifstream which means it can create files, write information to files, and read information from files.

Opening a File

A file must be opened before you can read from it or write to it. Either ofstream or fstream object may be used to open a file for writing. And ifstream object is used to open a file for reading purpose only.

Following is the standard syntax for open() function, which is a member of fstream, ifstream, and ofstream objects.

void open(const char *filename, ios::openmode mode);

Here, the first argument specifies the name and location of the file to be opened and the second argument of the open() member function defines the mode in which the file should be opened.

Mode Flag	Description
ios::app	Append mode. All output to that file to be appended to the end.
ios::ate	Open a file for output and move the read/write control to the end of the file.
ios::in	Open a file for reading.
ios::out	Open a file for writing.
ios::trunc	If the file already exists, its contents will be truncated before opening the file.

Closing a File

When a C++ program terminates it automatically flushes all the streams, release all the allocated memory and close all the opened files. But it is always a good practice that a programmer should close all the opened files before program termination.

Following is the standard syntax for close() function, which is a member of fstream, ifstream, and ofstream objects.

void close();

Writing to a File

While doing C++ programming, you write information to a file from your program using the stream insertion operator (<<) just as you use that operator to output information to the screen. The only difference is that you use an ofstream or fstream object instead of the cout object.

Reading from a File

You read information from a file into your program using the stream extraction operator (>>) just as you use that operator to input information from the keyboard. The only difference is that you use an ifstream or fstream object instead of the cin object.

```
#include <fstream>
#include <iostream>
using namespace std;

int main () {
    char data[100];

    // open a file in write mode.
    ofstream outfile;
    outfile.open("afile.dat");

    cout << "Writing to the file" << endl;
    cout << "Enter your name: ";
    cin.getline(data, 100);

    // write inputted data into the file.
    outfile << data << endl;

    cout << "Enter your age: ";
    cin >> data;
    cin.ignore();

    // again write inputted data into the file.
    outfile << data << endl;

    // close the opened file.
    outfile.close();

    // open a file in read mode.
    ifstream infile;
    infile.open("afile.dat");

    cout << "Reading from the file" << endl;
    infile >> data;
```

```
// write the data at the screen.  
cout << data << endl;  
  
// again read the data from the file and display it.  
infile >> data;  
cout << data << endl;  
  
// close the opened file.  
infile.close();  
  
return 0;  
}
```